

Python Fundamentos

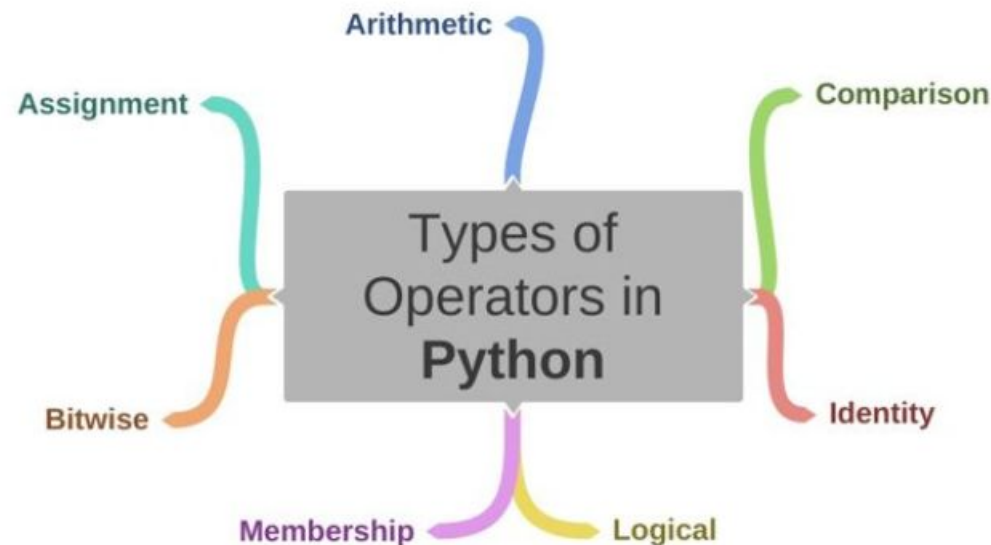
Tipos operadores

1



Una expresión une operandos y operadores según reglas sintácticas sencillas, de manera análoga a las reglas utilizadas en operaciones aritméticas. Además, puede incorporar palabras clave del lenguaje y signos de puntuación. Un operador es un símbolo que indica la operación a realizar sobre los operandos (datos) involucrados.

Por otro lado, una declaración o comando consiste en un conjunto de expresiones que posibilitan la ejecución de una acción específica.



Operadores aritméticos

2



Los operadores aritméticos en Python son símbolos que realizan operaciones matemáticas entre variables o valores. Aquí se exponen algunos ejemplos de los operadores aritméticos principales en Python:

Operador	Descripción	Ejemplo	Salida Esperada
+	Suma dos valores.	5 + 3	8
-	Resta el valor de la derecha.	10 - 4	6
*	Multiplica dos valores.	3 * 4	12
/	Divide el valor de la izquierda por el valor de la derecha, devolviendo un valor en punto flotante.	15 / 3	5.0
//	Divide y redondea hacia abajo al número entero más cercano.	17 // 3	5
%	Devuelve el resto de la división.	17 % 3	2
**	Eleva el valor de la izquierda a la potencia del valor de la derecha.	2 ** 3	8

Operadores comparación

3



Los operadores de comparación en Python se utilizan para comparar dos valores y determinar la relación entre ellos. Estos operadores devuelven un valor booleano (True o False) según si la comparación es verdadera o falsa.

Operador	Descripción	Ejemplo	Salida Esperada
<code>==</code>	Igualdad	<code>5 == 5</code>	True
<code>!=</code>	Desigualdad	<code>5 != 3</code>	True
<code>></code>	Mayor que	<code>7 > 4</code>	True
<code><</code>	Menor que	<code>2 < 6</code>	True
<code>>=</code>	Mayor o igual que	<code>5 >= 5</code>	True
<code><=</code>	Menor o igual que	<code>3 <= 4</code>	True

Operadores asignación

4



Los operadores de asignación en Python se utilizan para asignar valores a variables y actualizar su contenido. A continuación podemos ver una explicación de los operadores de asignación más comunes:

Operador	Descripción	Ejemplo	Salida Esperada
=	Asignación	x = 10	10
+=	Adición y asignación	x += 3	13
-=	Sustracción y asignación	x -= 2	11
*=	Multiplicación y asignación	x *= 4	44
/=	División y asignación	x /= 2	22
//=	División entera y asignación	x //= 3	7
%=	Módulo y asignación	x %= 5	2
**=	Exponente y asignación	x **= 2	4



Operadores lógicos



Los operadores lógicos en Python se utilizan para combinar o modificar expresiones lógicas. Estos operadores son fundamentales para la lógica de control de flujo y toma de decisiones en Python. Pueden combinarse para crear expresiones lógicas más complejas y son esenciales en estructuras condicionales y bucles. Además, permiten realizar evaluaciones lógicas en situaciones donde se necesitan decisiones basadas en la verdad o falsedad de las condiciones.

Operador	Descripción	Ejemplo	Salida Esperada
and	Devuelve True si ambas expresiones son verdaderas; de lo contrario, devuelve False.	<code>(5 > 3) and (7 < 10)</code>	True
or	Devuelve True si al menos una de las expresiones es verdadera; devuelve False si ambas son falsas.	<code>(5 > 3) or (7 > 10)</code>	True
not	Devuelve True si la expresión es falsa, y viceversa. Se utiliza para negar el valor de una expresión.	<code>not (5 > 10)</code>	True

Operadores
pertenencia

6



Se utilizan para verificar si un valor está presente en una secuencia de datos como son las listas, tuplas, diccionarios y strings.

Operador	Descripción	Ejemplo	Salida Esperada
in	Devuelve True si un valor está presente en una secuencia y False si no lo está..	lista = [1, 2, 3, 4, 5] resultado = 3 in lista	True
not in	Este operador devuelve True si un valor no está presente en una secuencia y False si está presente.	cadena = "Hola, Mundo" resultado = 'x' not in cadena	True

Operadores bit a
bit

7



Existe un tipo de operadores que son utilizados para realizar operaciones a nivel de bits. Estas operaciones manipulan los bits individuales de los números y son especialmente útiles cuando se trabaja con código de bajo nivel o se necesitan ciertas optimizaciones en el manejo de datos.

Operador	Descripción	Ejemplo	Salida Esperada
&	AND a nivel de bits	w = 5 w = w & 3	En binario, 5 es 101 y 3 es 011. Después de la operación AND a nivel de bits, obtenemos 001, que es 1 en decimal.
 	OR a nivel de bits	x = 5 x = x 3	En binario, 5 es 101 y 3 es 011. Después de la operación OR a nivel de bits, obtenemos 111, que es 7 en decimal.
^	XOR a nivel de bits	z = 5 z = z ^ 3	En binario, 5 es 101 y 3 es 011. Después de la operación XOR a nivel de bits, obtenemos 110, que es 6 en decimal.
<<	Desplazamiento a la izquierda	r = 4 r = r << 2	En binario, 4 es 100. Después de desplazar a la izquierda por 2 bits, obtenemos 10000, que es 16 en decimal.
>>	Desplazamiento a la derecha	s = 8 s = s >> 1	En binario, 8 es 1000. Después de desplazar a la derecha por 1 bit, obtenemos 100, que es 4 en decimal.
~	NOT	y = 5 y = ~y	El resultado será -6. Esto se debe a que la representación binaria de 5 es 0000 0101, y al aplicar la operación de inversión a nivel de bits, obtenemos 1111 1010 en binario, que es el equivalente a -6 en el sistema de complemento a dos.

Operadores de
identidad

8



Los operadores de identidad se utilizan para verificar si dos variables apuntan al mismo objeto en memoria o si un valor pertenece a una clase o tipo.

Operador	Descripción	Ejemplo	Salida Esperada
is	Pertenencia a clase o tipo	X = 5 type(x) is int	True
is not	No pertenencia a clase o tipo	X = 5.3 type(x) is not float	False

¡Gracias!